SMART INDIA HACKATHON · TECHNICAL PROPOSAL

JEEVAN

Adding machine learning to ambulance dispatch

Can it be implemented? Yes. Here is exactly where, and how.

- FastAPI + SvelteKit prototype · Yelahanka / BMSIT corridor
- Stack already includes scikit-learn, XGBoost, NumPy, pandas
- Keep Dijkstra / TomTom routing. Use ML to score, rank, and correct.

Contents

One-line answer. Implementable. Do not replace routing with a neural net. Plug a trained model into triage, ETA correction, and hospital ranking — the three places the current pipeline still uses rules and constants.

1. Verdict

Machine learning can be added to JEEVAN. The useful work is not replacing NetworkX Dijkstra or TomTom / OSRM with a neural net. It is plugging a trained model into the gaps the current pipeline still fills with checkbox rules and fixed multipliers.

The Python stack already lists scikit-learn, xgboost, numpy, and pandas in `backend/requirements.txt`. `ortools` is also declared and unused — that is an assignment solver, not ML, and should stay that way for multi-call matching.

There is already a stub at `backend/app/services/ml_triage.py`. It imports XGBoost, then ignores it and keyword-matches "fire" / "accident". `ARCHITECTURE.md` describes a Random Forest ETA pipeline that is not in the current code. The "AI" on the request page is NVIDIA LLMs for chat and report analysis — generative AI, not a trained dispatch model — and that analysis is never sent to `POST /tracking/dispatch`.

> So the question is not "can we add ML?" It is "which model actually improves this prototype, given there is almost no real trip history?" The rest of this document answers that with the current code as the source of truth.

2. What the system does today

Dispatch is optimisation plus heuristics. A staff or patient request on `/request` collects a description, narrative, age group, optional medical-report image, and four history flags. Only the flags and a hardcoded BMSIT coordinate reach the backend.

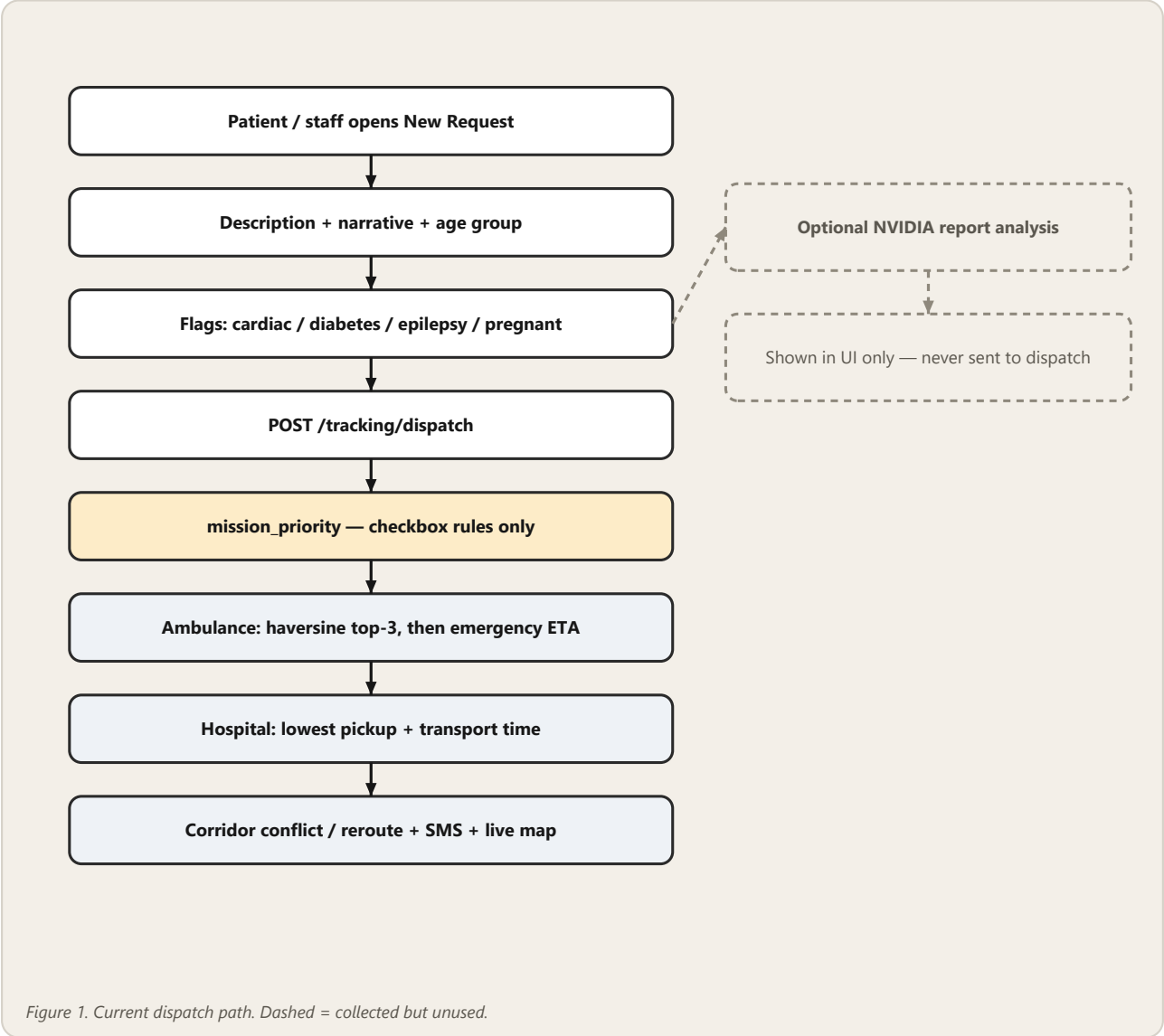

Figure 1. Current dispatch path. Dashed = collected but unused.

Ambulance pick (`dispatch_optimizer._pick_ambulance`) shortlists three units by haversine distance, then scores them with an emergency route. Hospital pick (`optimize_hospital_dispatch`) is the lowest pickup-plus-transport time. Beds and specialisations are stored on each hospital seed in `fleet.py` and returned in the candidate list, but they do not change who wins. Priority is `corridor.mission_priority`: cardiac=5, pregnant=4, epilepsy=3, diabetes=2, else 1. ETA is live routing × a fixed 0.70 emergency factor × 1.08 if raining. `_peak_traffic_factor` exists and is unused. Confidence is a heuristic gap between the winner and runner-up, not a model probability.

3. Gaps that ML should fill

Piece	Current behaviour	Unused / stub
Priority	Four medical checkboxes only	Free-text, age, and report analysis never change dispatch
Ambulance	Nearest 3 by distance, then fastest emergency route	No learned correction of that ETA
Hospital	Fastest ETA only	available_beds and specializations stored, not scored
ETA	Router seconds × 0.70 × optional rain 1.08	_peak_traffic_factor unused
ML file	Dummy keyword matcher in ml_triage.py	XGBoost never trained or loaded
Persistence	dispatch_cases / dispatches tables	Not enough real trip history to train on

4. Where ML belongs (and where it does not)

Treat ML as a scorer sitting in front of the optimiser you already have. Live routing, corridor occupancy, and hard medical constraints stay deterministic. That is also the honest story for judges: you are not claiming a self-driving ambulance.

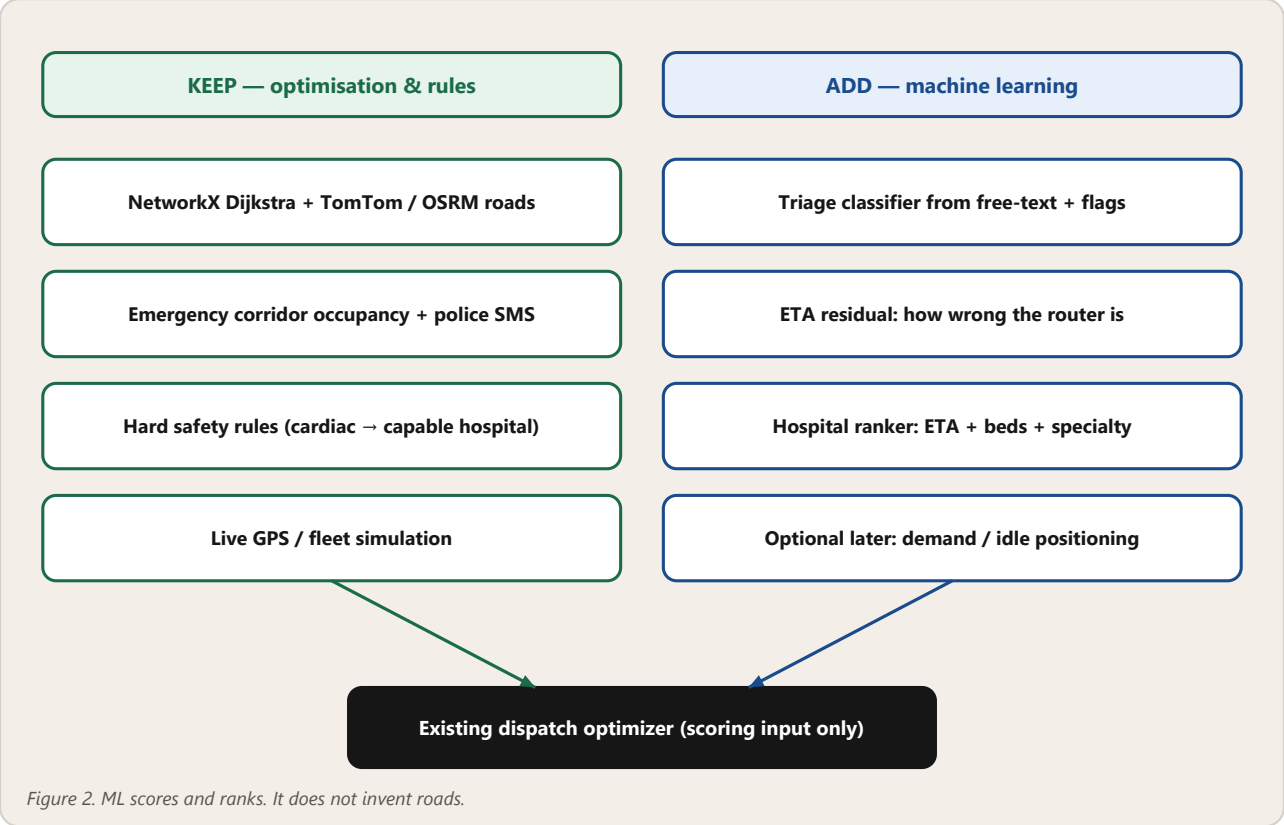

Figure 2. ML scores and ranks. It does not invent roads.

Keep as-is	Add ML
Which roads to drive (Dijkstra / TomTom / OSRM)	How severe the call is (triage)
Corridor conflict + police / rescue SMS	How wrong the router ETA is for this hour / weather
Cardiac must reach Cardiac / ICU / Trauma if a capable hospital is nearby	Rank hospitals by ETA + beds + specialty match
Live GPS simulation	Demand / idle positioning (later only)

> Do not train a model to invent routes. The graph already does that. ML should score, rank, or correct numbers the optimiser already produces.

5. Target pipeline after ML

The request page already collects everything a triage model needs. The dispatch endpoint already accepts `notes` and an optional `priority` override. The missing piece is a load-once model call between those two.

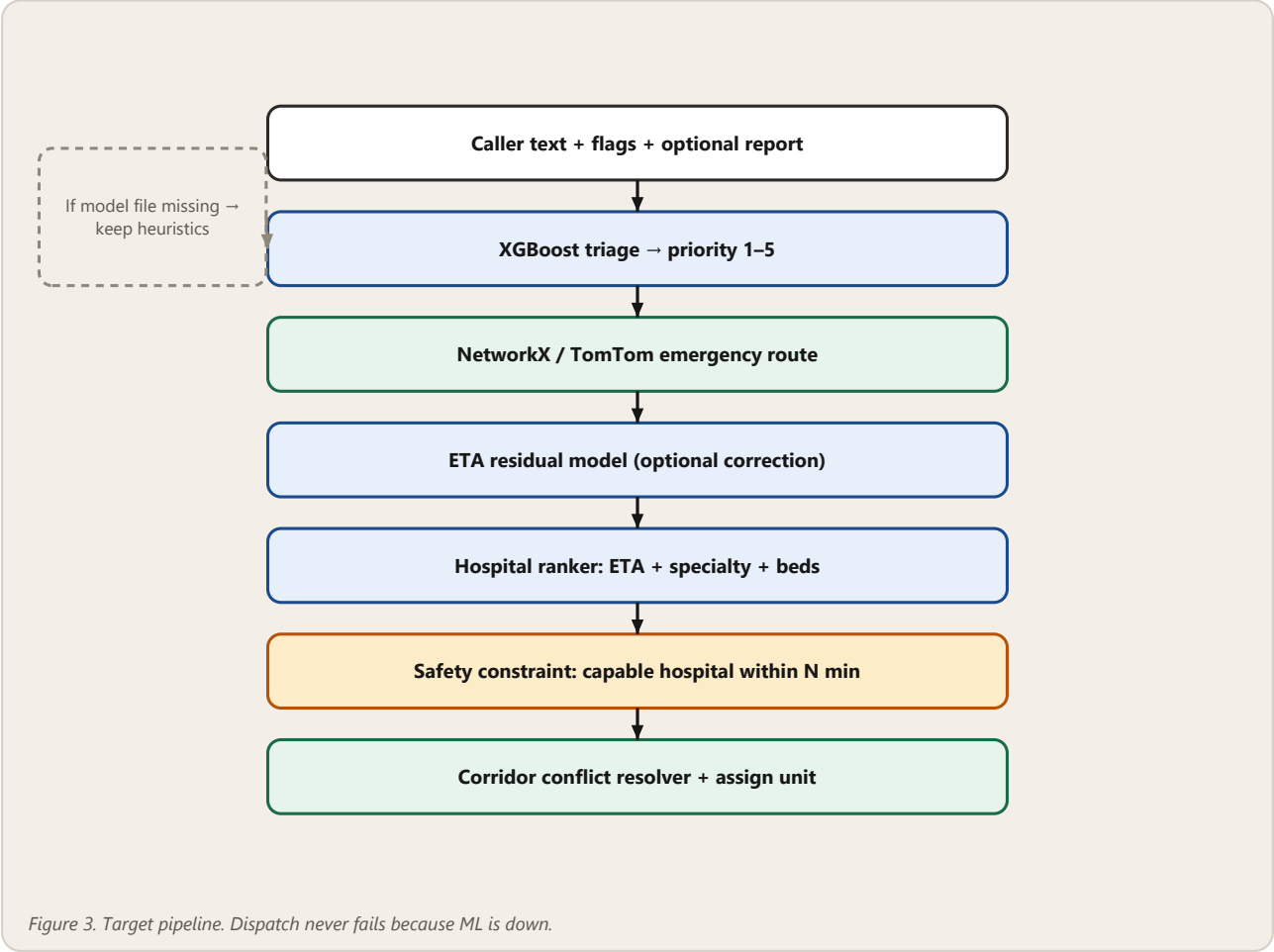

Figure 3. Target pipeline. Dispatch never fails because ML is down.

Runtime rule. Load models at API process start. If a `.ubj` or `.json` file is missing, keep today's heuristics. Dispatch must never 500 because ML is down.

Do not call NVIDIA on the hot path for every SOS. LLMs are slow, costly, and non-deterministic. Use them only to enrich features (a report summary), then let XGBoost decide. That keeps the "AI analysis" button as a feature source, not as the dispatcher.

6. Model 1 — incident triage (ship first)

Problem. Description, age, and report analysis never change dispatch. Priority is only four checkboxes. A "severe bleeding, unresponsive" narrative is treated the same as a stable fever if the flags are empty.

Model. Gradient boosting (XGBoost — already a dependency) classifying LOW / MEDIUM / HIGH / CRITICAL, mapped onto the existing 1–5 `mission_priority` scale so corridor conflict resolution does not need a new type.

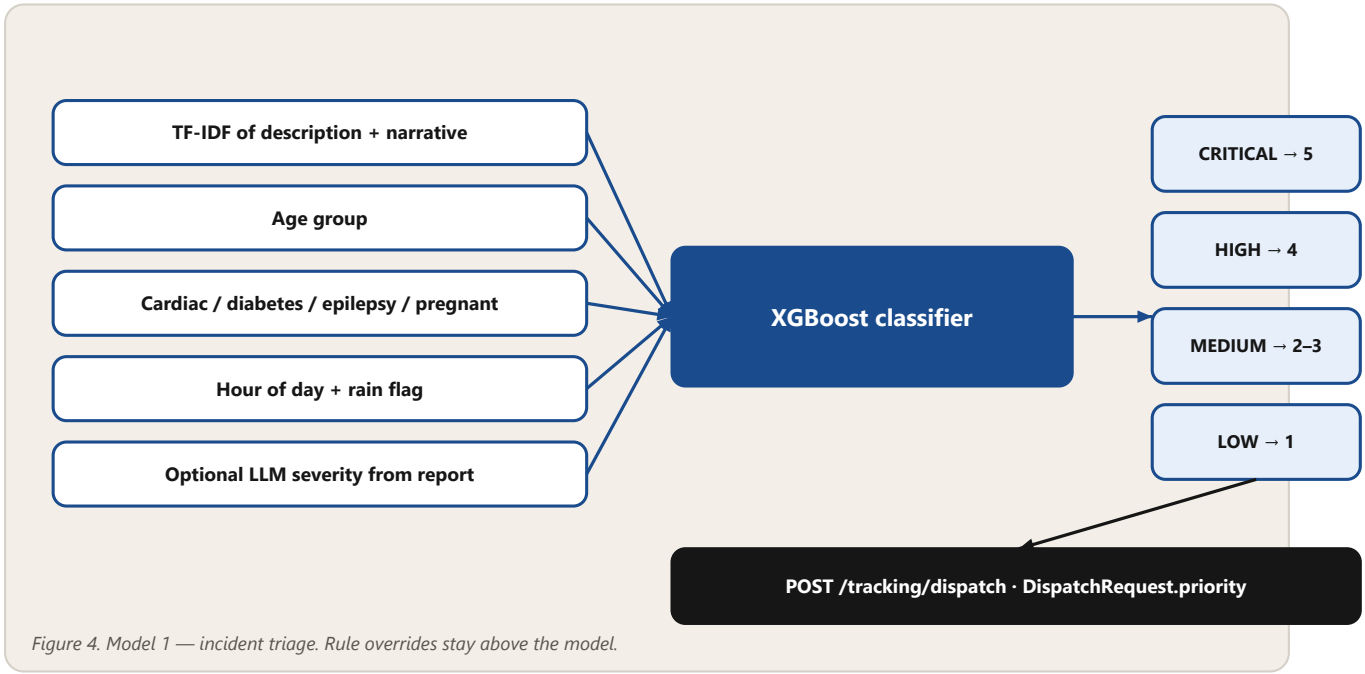

Figure 4. Model 1 — incident triage. Rule overrides stay above the model.

Features

- Text: TF-IDF (or a small embedding) of description + narrative
- Age group already collected on the request page (infant → elderly)
- Cardiac / diabetes / epilepsy / pregnant flags
- Hour of day and the existing is_raining flag
- Optional: parsed severity from /ai/analyze-report — LLM as a feature, not the decision

Labels and training (hackathon-honest)

There is no 108-call archive in this repo. Judges will accept a synthetic set if you can defend the rules, show held-out accuracy, and keep a fallback. Example label policy:

Pattern in caller text / context	Label	Priority
Unresponsive, not breathing, severe bleeding, chest pain + collapse	CRITICAL	5
Accident / fracture, conscious; major trauma flags	HIGH	4
Fever, fall, stable vitals language; chronic flags only	MEDIUM	2–3
Non-urgent transport language	LOW	1

Train offline in `backend/app/ml/train_triage.py`, dump `backend/app/ml/models/triage.ubj`, load once in `ml_triage.py` replacing DummyTriageModel. Safety: keyword / rule overrides stay above the model (unresponsive → always CRITICAL). On the request page, show a severity chip with confidence before Auto-assign, and send the predicted priority in the dispatch body.

> Why this is the first model. Highest demo value, smallest data need, and it completes a file the repo already pretends exists. It also matches what ARCHITECTURE.md promised ("rule-based triage → ML severity adjustment").

7. Model 2 — ETA residual (technical depth)

Problem. TomTom / OSRM duration × 0.70 is a guess. Real ambulance time in North Bangalore depends on hour, rain, corridor occupancy, and road class. The peak-hour factor is written and never applied.

Model. Regression (XGBoost or sklearn Random Forest) predicting a residual in seconds:

```
actual_travel_s - router_duration_s
```

At inference: `eta = router_duration * 0.70 * rain_factor + model.predict(features)`. Wire this into `_apply_emergency_eta` / `optimize_hospital_dispatch` so hospital ranking uses the corrected minutes. If the model file is missing, keep the current constants.

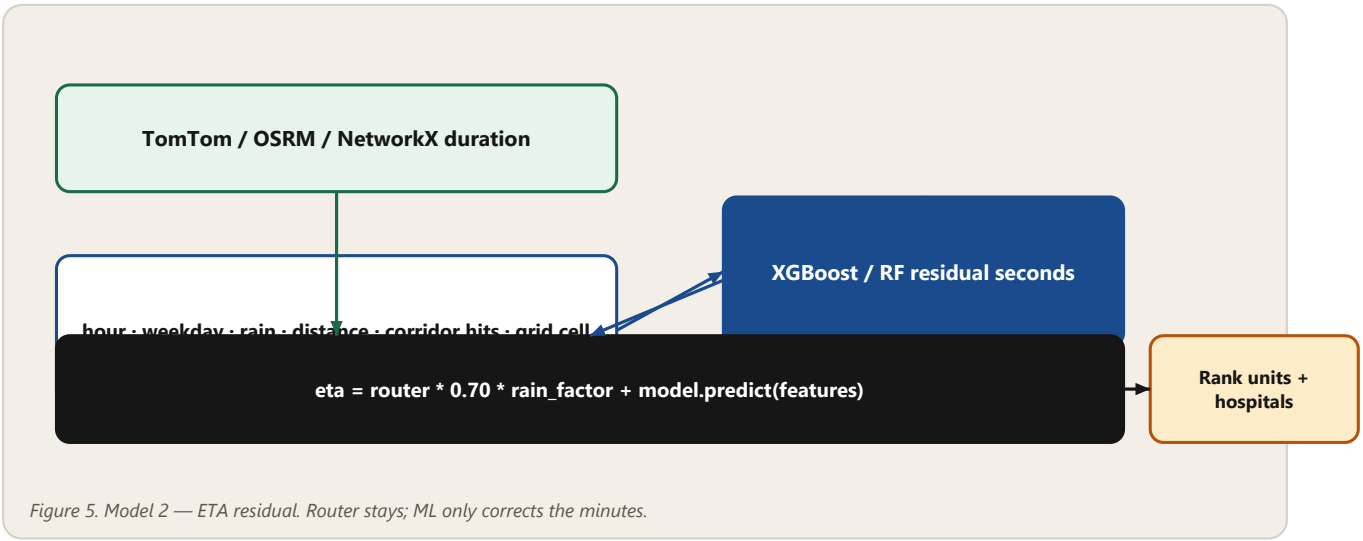

Figure 5. Model 2 — ETA residual. Router stays; ML only corrects the minutes.

Data problem. Simulated fleet ticks are not ground truth. For SIH: (1) log every dispatch — predicted ETA, route source, weather, hour, hospital, ambulance; (2) when a simulated trip completes, log actual seconds from phase_started_at → complete; (3) optionally synthesise thousands of origin–destination pairs with a noisy "truth" (router time × random 0.85–1.35 by hour and rain). Say "trained on simulated Yelahanka missions + router residuals," not "production EMS model." This is the piece ARCHITECTURE.md called `ml_engine.py`.

8. Model 3 — hospital ranking

Problem. Lowest ETA can send a cardiac patient to Yelahanka Government Hospital if it is two minutes closer than Aster CMI or M.S. Ramaiah — both of which list Cardiac / Trauma / ICU on the seed data. Beds never enter the score.

You do not need a neural net first. A weighted score on the existing candidate list is enough, and it is easier to defend:

```
score = w1 * eta_min + w2 * (0 if specialty matches else 15) + w3 * max(0, 4 - available_beds) + w4 * (0 if ICU else 10)
```

If you want ML later: an XGBoost ranker on synthetic "good destination" labels (cardiac → Cardiac/ICU even if slightly slower; pediatric + child → pediatric; trauma + HIGH → trauma with beds). Either way, keep a hard constraint in front of the scorer.

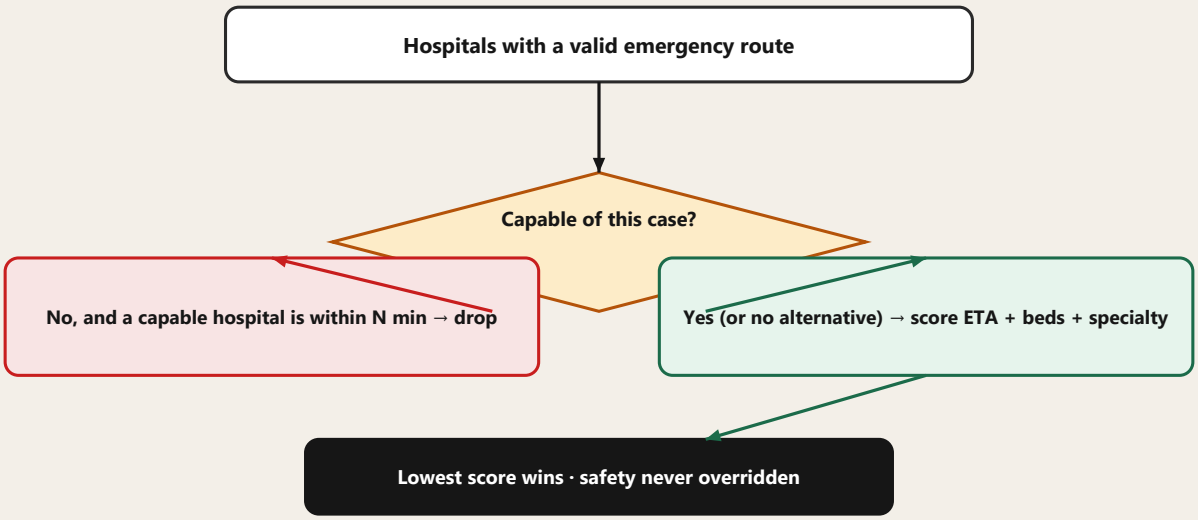

Figure 6. Model 3 — hospital ranking. Cardiac never skips ICU/Trauma if one is nearby.

Hard constraint. Never send cardiac to a hospital with no Cardiac / ICU / Trauma if a capable one is within N minutes. ML reranks the feasible set. It does not override safety. That sentence belongs on the demo slide.

9. Files, runtime rules, and what not to do

Suggested layout

```
backend/app/ml/train_triage.py generate synthetic data, train, dump model
backend/app/ml/train_eta.py optional, phase C
backend/app/ml/models/triage.ubj committed artefact for the demo
backend/app/services/ml_triage.py load-once + predict(); delete dummy keywords
backend/app/api/tracking.py call triage, set priority
frontend/src/routes/request/+page.svelte severity chip before Auto-assign
```

Later, not ML

- OR-Tools assignment when several SOS events arrive at once — already in requirements, unused. Use it instead of a neural matcher.
- Demand heatmap / idle positioning needs weeks of logs; fake Bangalore peak patterns only if time remains.
- Vitals deterioration — tick_vitals is random noise; a small "unstable" classifier is an easy demo and weak science. Do not lead with it.

A stub that imports unused XGBoost looks worse than no ML file. Either train ml_triage.py or remove the dead import. Judges will open that file.

10. Phased plan and honest constraints

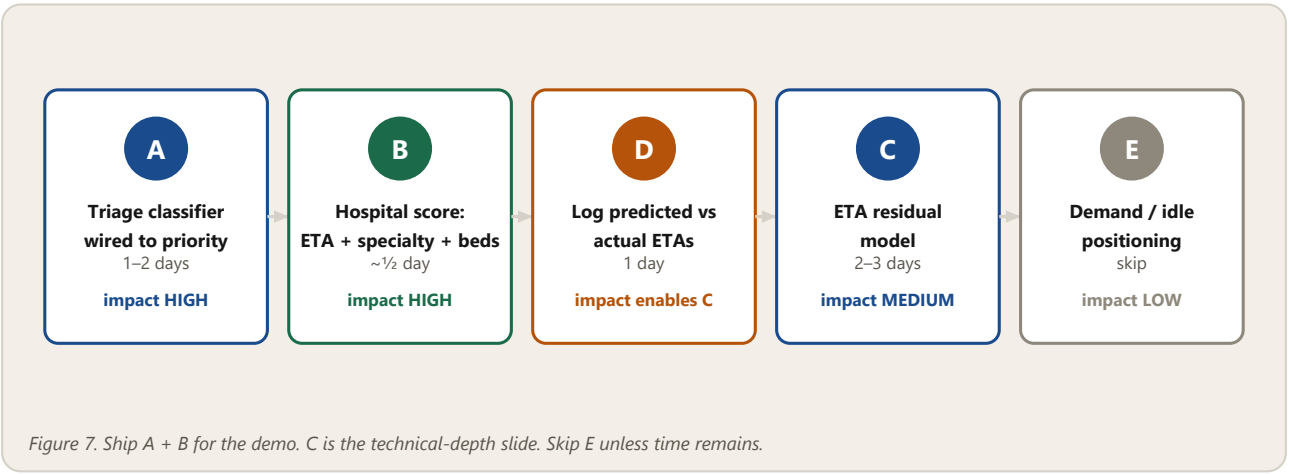

Figure 7. Ship A + B for the demo. C is the technical-depth slide. Skip E unless time remains.

Phase	What	Effort	Demo impact
A	Text + flags → XGBoost → priority on dispatch	1–2 days	High — "AI triage" becomes real
B	Hospital score: ETA + specialty + beds (rules first)	~½ day	High — cardiac goes to Aster / Ramaiah
D	Log predicted vs actual on dispatch_cases	1 day	Needed before C is real
C	Synthetic ETA residual model on logged trips	2–3 days	Medium-high — technical depth
E	Demand / idle positioning	skip unless time	Low for SIH

For a hackathon, A + B is the honest win. C is the technical-depth slide. Do not start with deep learning or reinforcement learning for routing.

Constraints to say out loud

- No real 108/EMS dataset in the repo. Synthetic training is acceptable if you show the feature list, held-out accuracy, and the fallback path.
- Simulated fleet time is not Bangalore traffic. Caption C as simulated Yelahanka residuals, not a production travel-time model.
- Medical ML is high-stakes. Rule overrides (unresponsive → CRITICAL; cardiac → capable hospital) stay above the model.
- The current "AI" features (Nemotron chat, Tavus intake, vision report analysis) can stay as product UX. They are not the dispatch brain.

Bottom line

Implementable, and the hooks already exist: `ml_triage.py`, sklearn / xgboost, `priority` on dispatch, unused hospital beds and specialisations. Start by making triage a real classifier and hospital pick specialty-aware. Add ETA residual ML only after those two are visible in the UI. Keep Dijkstra. Keep the corridor. Keep the safety rules.

This document describes the JEEVAN prototype in this repository. It is not a medical device specification and must not be used to dispatch real emergency vehicles without a clinically validated protocol and live EMS data.